Information theory

Emanuele Di Prima

March 2026

Contents

1 Introduzione

Il ruolo fondamentale della teoria dell'informazione e compressione dati fornisce i concetti fondamentali, le tecniche e gli strumenti per rappresentare, processare e trasmettere in modo sicuro ed efficiente le informazioni. Fondamentalmente:

- **Information theory:** stabilisce i principi per rappresentare e quantificare l'*informazione* contenuta nei dati emessi da una sorgente.
- **Data compression:** descrive le tecniche per ridurre la dimensione dei dati (i bits) per migliorare la loro trasmissione e salvataggio.

Tutto questo è possibile solo perché la rappresentazione della realtà porta con se molta ridondanza, per cui identificandola si può eliminare ottimizzandone la rappresentazione. Questi concetti della teoria dell'informazione sono alla base della crittografia; anche per il ML (Machine Learning) comprimendo i bit dei dataset i modelli occupano meno spazio e sono computazionalmente più efficienti.

1.1 Cos'è l'informazione?

Nell'informatica, lo scambio di informazioni si riferisce al trasferimento di dati tra un trasmittente e un ricevente (non sono necessariamente persone fisiche) e la **misura** del contenuto informativo di un messaggio viene rappresentato da quanto questo riesce a ridurre l'incertezza che si aveva prima di riceverlo. In altre parole, si presuppone che prima di ricevere un **messaggio** fossimo incerti su un determinato argomento e dopo averlo ricevuto siamo venuti a conoscenza di informazioni aggiuntive che hanno ridotto questa incertezza.

Definizione 1.1. L'informazione è la misura della riduzione dell'incertezza ed è quantificata dall'ammontare di conoscenza "guadagnata" da una osservazione o un messaggio. Quando un messaggio o un evento è altamente improbabile porta con se molta informazione poiché riduce l'incertezza maggiormente rispetto ad un evento molto probabile.

1.1.1 Criteri di una misura dell'informazione accettabile

I criteri fondamentali per una misura del **contenuto informativo** accettabile sono:

- **Dipendenza dalla probabilità:**
  1. Un'informazione associata ad un evento dovrebbe essere collegata alla **probabilità delle sue occorrenze**.
  2. Più un evento è raro e più contenuto informativo porta con se.
- **Funzione monotonica e continua:**
  1. Il contenuto informativo di un evento dovrebbe **aumentare al diminuire della probabilità**. Applicato agli assiomi di Shannon, questo concetto stabilisce anche che, in un sistema composto da eventi tutti equiprobabili, un numero maggiore di eventi possibili deve corrispondere a una maggiore incertezza complessiva.
  2. La quantità di informazione deve **variare in modo continuo** al variare della probabilità dell'evento. Dal punto di vista formale, ciò richiede che l'entropia sia una funzione continua rispetto alla probabilità nell'intervallo reale. In termini pratici, questo significa che un cambiamento piccolissimo nella probabilità di un evento non deve causare un "salto" improvviso e sproporzionato nel calcolo dell'informazione, ma una variazione graduale.
- **Eventi indipendenti:** Se due eventi sono statisticamente indipendenti, il contenuto informativo totale sarà dato dalla **somma** dei contenuti informativi dei singoli eventi.

1.1.2 Esempio

Un esempio, supponiamo di avere un urna con 100 palline numerate ognuna con la stessa probabilità di essere estratta. La probabilità di estrarne una è $\frac{1}{100}$. Ora supponiamo di tirare una moneta e la probabilità che esca testa o croce è $\frac{1}{2}$. Se venisse comunicato il messaggio contenente cosa è stato estratto o se è uscito testa o croce, quale contenuto ha **maggiore informazione**?

Il messaggio dell'estrazione della pallina dall'urna è quello che ha il maggior contenuto informativo rispetto al lancio della moneta. Questo perché nella scelta dell'urna c'era molta più incertezza avendo una probabilità più bassa di estrarre ogni singola pallina. Per cui, più sono le possibilità è più incertezza c'è prima di ricevere il messaggio. Il contenuto informativo del lancio della moneta è il minimo possibile visto che ci sono solo due possibilità equiprobabili; il contenuto informativo è precisamente 1 bit.

1.2 Sistema di comunicazione

Secondo Shannon, un qualsiasi sistema di comunicazione consiste in 5 fasi:

- **Sender:** è la sorgente del messaggio che deve essere comunicato.
- **Trasmitter:** usato per codificare ed inviare il messaggio attraverso il canale di comunicazione.
- **Canale di comunicazione:** il mezzo usato per trasmettere il messaggio.
- **Receiver:** ciò che riceve e decodifica il messaggio.
- **Destination:** il soggetto che doveva ricevere il messaggio.

Shannon, inoltre, ha introdotto due ipotesi:

- La trasmissione di simboli attraverso un canali di comunicazione è un fenomeno discreto che impiega una certa quantità di tempo diversa da zero.
- Il canale di comunicazione è affetto da **sorgenti di rumore** le quali possono modificare il contenuto informativo del messaggio in maniera impredicibile a priori. La trasformazione potrebbe anche corrompere il messaggio stesso.[1]

Nel processo di comunicazione la misura della quantità di informazione viene espressa attraverso i **bit**, l'unità di misura fondamentale dell'informazione, che permette una quantificazione esatta di quanta informazione si sta trasmettendo.

1.3 Misura dell'informazione

Una **sorgente** è caratterizzata da un alfabeto intrinseco, ogni elemento dell'alfabeto può essere di qualsiasi natura come lettere, numeri o eventi. La sorgente, come già accennato, è **discreta** e ad ogni unità di tempo viene prodotto un singolo elemento di questo alfabeto con una sua probabilità. Questa probabilità viene mappata come $\pi : A \to [0, 1]$ per cui una sorgente discreta S può essere rappresentata da:

$$S = \begin{pmatrix} a_1 & a_2 & ... & a_m \\ p_1 & p_2 & ... & p_m \end{pmatrix} \tag{1.1}$$

dove deve valere $\sum_{a \in A} \pi(a) = 1$. Ogni colonna di questa matrice rappresenta un elemento con la sua probabilità associata; per cui, per esempio, al simbolo a_1 è associata una probabilità p_1.

1.3.1 Auto-informazione

Definizione 1.2. Dato un simbolo a_i associato ad una sorgente S di un determinato alfabeto A, si definisce **auto informazione** di a_i:

$$I(a_i) = \log_b \left(\frac{1}{p_i} \right) \tag{1.2}$$

[1] Il corso tratta solo le teorie senza rumore.

rappresenta la **misura quantitativa del contenuto informativo** associato ad uno specifico simbolo (o evento) emesso da una sorgente. La base dell'algoritmo può essere:

- $b = 2$: rappresenta la misura di un **BIT** (Binary DigIT).
- $b = e$: rappresenta la misura di un **NAT**.
- $b = 10$: rappresenta la misura di **HARTLEY**.

Dalla definizione precedente si può assumere che se la probabilità di un simbolo è 1 allora il contenuto informativo sarà pari a 0. Invece, se la probabilità tende a 0 allora il contenuto informativo tenderà ad infinito. Questo significa che meno probabile è un evento più sarà l'informazione che possiede. Formalizzando le proprietà dell'auto-informazione che derivano da quelle dei logaritmi:

- $I(a_i) \geq 0$ per $0 \leq p_i \leq 1$
- $I(a_i) \to 0$ per $p_i \to 1$
- $I(a_i) > I(a_j)$ per $p_i < p_j$
- Dati due simboli indipendenti a_i e a_j, abbiamo:
  - $\pi(a_i a_j) = p_i \cdot p_j$
  - $I(a_i a_j) = I(a_i) + I(a_j)$

Esempio Se consideriamo il caso binario ($b = 2$) allora $I_i = \log_2(\frac{1}{p_i})$ e per una sorgente S di due simboli equiprobabili:

$$S = \begin{pmatrix} a_1 & a_2 \\ \frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

il contenuto informativo sarà $I_1 = I_2 = \log_2 2 = 1[bit]$. Come si nota, serve un bit per distinguere due messaggi equiprobabili.

1.3.2 Entropia di Shannon

L'entropia può essere interpretata come la **quantità di informazione media** che ogni simbolo emesso da una sorgente trasporta o, in termini più formali, in media il numero minimo di bit necessari per codificare un simbolo emesso da una sorgente. Shannon attraverso un approccio assiomatico, stabilì preliminarmente che qualsiasi funzione matematica candidata a misurare il contenuto informativo dovesse necessariamente rispettare alcuni criteri fondamentali, dettati dalla logica e dalla coerenza matematica. Questi cinque assiomi impongono che la misura dell'informazione debba essere continua nel dominio delle probabilità, normalizzabile per definire l'unità di misura, invariante rispetto a come le probabilità vengono etichettate (permutazione), monotonicamente crescente al crescere del numero di eventi equiprobabili e, infine, coerente e additiva quando gli eventi vengono raggruppati in scelte successive

Teorema 1.1. Consideriamo una sorgente S come nella (**??**), allora **l'entropia** di S è definita come:

$$H(S) = C \sum_{i=1}^{n} p_i \log_b \left(\frac{1}{p_i}\right) = C \sum_{i=1}^{n} p_i \cdot I(a_i) \tag{1.3}$$

assumendo che l'unità di misura sia il bit, per cui il logaritmo è espresso in base 2 ed la costante $C \geq 0$. Solitamente C dipende dall'unità di misura scelta (ovvero la base del logaritmo) e in genere è uguale a 1.

Dimostrazione. Secondo Shannon, è ragionevole assumere che una funzione

$$H(S) = H_n(p_1, p_2, \ldots, p_n)$$

che misura il contenuto informativo medio prodotto da una sorgente rispetta le seguenti proprietà (assiomi):

1. H_2 è una funzione continua di p nell'intervallo $[0, 1]$

2. $H_2(\frac{1}{2}, \frac{1}{2}) = 1$

3. (Invarianza alle permutazioni) $H_n(\sigma(p_1, p_2, ..., p_n)) = H_n(p_1, p_2, ..., p_n)$ l'entropia non varia rispetto alle permutazioni delle probabilità della sorgente. L'incertezza non dipende infatti da come sono state assegnate le probabilità ai singoli simboli.

4. Detta $A(n) = H_n(\frac{1}{n}, ..., \frac{1}{n})$ questa deve essere una funzione di n monotonica crescente. Infatti, se tutti gli eventi sono equiprobabili, maggiore è il numero di questi eventi e maggiore sarà l'incertezza (un contenuto informativo maggiore). **Esempio:** se ho dei simboli A, B, C con delle probabilità del $20\%, 50\%, 30\%$ rispettivamente se queste probabilità fossero del $50\%, 30\%, 20\%$ l'entropia sarebbe la stessa in entrambi i casi.

5. Se una scelta viene divisa in altre due scelte successive, il valore originale di H deve essere la somma pesata dei singoli valori di H, cioè deve valere:

$$H_n(p_1, p_2, \ldots, p_n) = H_{n-1}(p_1 + p_2, p_3, \ldots, p_n) + (p_1 + p_2) H_2\left(\frac{p_1}{p_1 + p_2}, \frac{p_2}{p_1 + p_2}\right) \tag{1.4}$$

   Esempio:[2] $H_3(\frac{1}{6}, \frac{1}{3}, \frac{1}{2}) = H_2(\frac{1}{2}, \frac{1}{2}) + \frac{1}{2} H_2(\frac{1}{3}, \frac{2}{3})$. Questo assioma può anche essere esteso al raggruppamento di k simboli:

$$\begin{aligned} H_n(p_1, p_2, \ldots, p_n) = {} & H_{n-k+1}(p_1 + \cdots + p_k, p_{k+1}, \ldots, p_n) \\ & + (p_1 + \cdots + p_k)\, H_k\left(\frac{p_1}{p_1 + \cdots + p_k}, \ldots, \frac{p_k}{p_1 + \cdots + p_k}\right) \end{aligned} \tag{1.5}$$

Finita la definizione degli assiomi è "facile" verificare che la funziona (**??**) li soddisfa:

1. Dall'assioma 5 deriviamo che $A(n \cdot m) = A(n) + A(m)$.

2. $A(n^k) = kA(n)$ per $k, n > 0$. Basta provarlo per induzione grazie alla precedente.

3. Dagli assiomi sappiamo che la funzione deve essere monotonica crescente, per cui l'unica funzione che soddisfa questa caratteristica e quelle precedenti è $A(n) = C \log_b(n)$ una funzione logaritmica con $n, C > 0$.

4. L'entropia di una scelta binaria diversa da quella equiprobabile:

$$H_2(p, p-1) = -C\left[p \log(p) + (1-p) \log(1-p)\right]$$

   da dimostrare per tutti i numeri razionali.

5. Estendiamo la dimostrazione precedente a tutti gli n per induzione e troviamo $H(S) = C \sum_{i=1}^{n} p_i \log_b\left(\frac{1}{p_i}\right)$.

□

[2] Per meglio comprendere questo assioma, lo spiego in modo più semplice. L'entropia del sistema non cambia anche se effettuiamo prima una decisione intermedia; quindi, raggruppare o meno le probabilità produce lo stesso risultato. Immaginiamo di avere tre eventi: A, B, C con le probabilità dell'esempio. Possiamo effettuare una scelta intermedia su un sottogruppo che rappresenta p_1 e p_2, distinto dal sottogruppo che rappresenta p_3. Pertanto, le probabilità per effettuare la scelta intermedia sono quelle descritte dall'assioma, ovvero $\frac{1}{2}$ per entrambi i casi (cioè $H_2(\frac{1}{2}, \frac{1}{2})$). Supponiamo ora di scegliere il sottogruppo formato da p_1 e p_2: in questo caso dobbiamo effettuare un'ulteriore scelta, poiché il gruppo è composto da più elementi. Le probabilità delle singole scelte saranno, rispettivamente, $\frac{1}{3}$ e $\frac{2}{3}$ (cioè $H_2(\frac{1}{3}, \frac{2}{3})$). Si nota che il prodotto tra la probabilità della scelta intermedia (del gruppo) e quella della scelta finale restituisce le probabilità iniziali. Facendo un esempio concreto, se i simboli sono gusti di gelato, ad esempio $A =$ Cioccolato, $B =$ Vaniglia, $C =$ Frutta, allora ci sono due modi per arrivare alla scelta finale. Il primo consiste nello scegliere direttamente, secondo le probabilità originarie. Il secondo consiste in una scelta composta: prima si seleziona un sottogruppo, ad esempio i gusti alla Crema (cioè non Frutta), e poi si sceglie all'interno del sottogruppo. La prima scelta avviene tra Crema e Frutta, ciascuna con probabilità $\frac{1}{2}$. Una volta scelta, ad esempio, Crema, si effettua una seconda scelta con probabilità condizionate dalla scelta intermedia.

1.3.3 Esempio

Vengono qui elencati alcuni esempi di calcolo di entropia. Consideriamo una sorgente S con i seguenti simboli e probabilità associate:

$$S = \begin{pmatrix} a_1 & a_2 & a_3 \\ \frac{1}{2} & \frac{1}{4} & \frac{1}{4} \end{pmatrix}$$

allora l'entropia di questa sorgente sarà:

$$H(S) = \frac{1}{2}\log_2(2) + \frac{1}{4}\log_2(4) + \frac{1}{4}\log_2(4) = \frac{3}{2}\text{bits/symbol}$$

come si nota, si può interpretare questa quantità come la media di informazione per simbolo o, come da definizione iniziale di informazione, la **quantità media di incertezza** che un osservatore ha prima di conoscere il contenuto di un messaggio.

Consideriamo una sorgente che emette tre simboli a, b e c con, rispettive, probabilità di 0.4, 0.3 e 0.3. In formula possiamo scrivere:

$$S = \begin{pmatrix} a & b & c \\ 0.4 & 0.3 & 0.3 \end{pmatrix}$$

allora:

$$H(S) = 0.4\log_2\left(\frac{1}{0.4}\right) + 0.3\log_2\left(\frac{1}{0.3}\right) + 0.3\log_2\left(\frac{1}{0.3}\right) = 0.84 \text{ bits/symbol}$$

1.4 Codici univocamente decodificabili

Solitamente, l'alfabeto di una sorgente non è trasmesso direttamente. Un encoder, prima di trasmettere, si occupa di codificare il messaggio trasformando i simboli del vecchio alfabeto in quelli del nuovo cosi da rendere agevole la loro trasmissione attraverso il canale di comunicazione. Una volta arrivato il messaggio deve essere decodificato per poter tornare al contenuto originale.

Definizione 1.3. Data una sorgente $S = \{s_1, s_2, \ldots, s_m\}$. Viene definita **funzione di codifica**[3] una funzione che mappa una sequenza degli elementi di S in una sequenza di un altro alfabeto $X = \{x_1, x_2, \ldots, x_m\}$, chiamato **alfabeto di codifica**.

Le **funzioni di codifica** possono essere rappresentate come tabelle di associazione dei simboli dell'alfabeto originale in quelli dell'alfabeto codificato. Ecco un esempio:

Simbolo	**Codifica**
s_1	0
s_2	11
s_3	00
s_4	01

questa tabella, come si nota, riporta delle **codifiche** tutte differenti fra loro. Un blocco di codifica si dice **non singolare** se ogni parola dell'alfabeto di codifica è distinta. Questa proprietà non è sufficiente infatti data la sequenza codificata 0011 ci sono due possibili combinazioni di decodifica: $s_3 s_2$ o $s_1 s_1 s_2$. La necessità di avere una proprietà più rigorosa è fondamentale.

Definizione 1.4. Un codice è detto **unicamente decodificabile** (UD) se ogni elemento della sequenza dell'alfabeto di codifica corrisponde ad una sola sequenza dell'alfabeto della sorgente. In pratica, la decodifica di ogni parola deve essere univoca.

[3] Nelle slide queste vengono definite in diversi modi. Qui vengono definite come funzioni, visto che lo sono a tutti gli effetti. Quindi quando si parla di **funzioni di codifica** si fa riferimento sia a code che a block code. Inoltre, verrà usato anche il termine **blocco di codifica** per riferirsi alla stessa cosa.

1.4.1 Algoritmo Sardinas-Patterson

Verificare che un codice generico sia UD è complesso, per questo motivo si utilizza il teorema di Sardinas-Patterson dal quale è possibile dedurre un algoritmo adatto alla verifica di questi codici. Il suo costo computazionale è $O(rL)$ dove r è il numero di simboli di codifica (codewords), o parole di una frase, e L è la somma delle lunghezze delle stringhe di simboli, o la lunghezza delle frasi.

Teorema 1.2. Dato un codice C basato su un'alfabeto A e consideriamo gli insiemi $S_0, S_1, S_2, \ldots$ generati nel seguente modo:

1. $S_0 = C$
2. $S_i = \{w \in A^* : \exists a \in S_0, \exists b \in S_{i-1} \implies a = bw \text{ o } b = aw\}$ con $i \geq 0$

dove A^* rappresenta l'insieme di tutte le stringhe di lunghezza finita generabili sull'alfabeto A, e l'insieme S_i rappresenta formalmente l'insieme dei "residui" o "suffissi" ricavati al passo i. Allora la condizione **necessaria e sufficiente** affinché C sia UD è:

$$\forall n > 0, S_0 \cap S_n = \emptyset$$

ovvero, nessuno degli insiemi dei residui generati corrisponde ad una codewords.

Il processo iterativo convergerà verso una delle seguenti possibili condizioni di stop:

- $\exists i > 0, S_0 \cap S_n \neq \emptyset$ allora il codice C non è UD.
- $\exists i > 0, S_i = \emptyset$ allora il codice C è UD.
- $\exists i, j > 0, S_i = S_j$, se esistono due insieme dei residui uguali allora il codice C è UD. In questo caso, è presente un ciclo infinito che comporta un ritardo di decodifica illimitato[4].

1.4.2 Esempio codice non UD

[5] Facciamo un esempio con il codice $C = \{a, c, ad, abb, bad, deb, bbdce\}$, la prima operazione è associare all'insieme base l'insieme di codifica, cioè $S_0 = C$, poi generando gli insieme dei residui secondo i seguenti passaggi:

- **Iterazione 1:** Per generare S_1 effettuo un confronto tra l'insieme base S_0 e S_{i-1} (che in questa prima iterazione coincide con S_0). Per ogni elemento dell'insieme base S_0, **cerco nell'altro insieme un codice diverso tale che uno dei due sia prefisso dell'altro** (la regola di ricerca verifica entrambe le occorrenze)[6]. Analizzando il primo codice di S_0, che è a, si osserva che esso è prefisso del codice ad; viene quindi estratto il residuo $w = d$ ed inserito nel nuovo insieme. Proseguendo l'analisi per il codice a, si nota che esso è prefisso anche del codice abb, generando così un secondo residuo $w = bb$. Questo processo di confronto incrociato viene eseguito per tutti i codici. Alla fine della prima iterazione, l'insieme generato è $S_1 = \{d, bb\}$.
- **Iterazione 2:** Per generare S_2 effettuo un confronto tra S_0 e S_1. In questo caso controllo ogni elemento di S_1, dove il primo è d, prefisso di $deb \in S_0$; viene quindi generato il residuo $w = eb$ ed inserito nel nuovo insieme. Il secondo elemento che controllo è bb questo è prefisso di $bbcde$ che produce come residuo $w = cde$. Quindi $S_2 = \{eb, cde\}$.

[4] Il messaggio deve essere ricevuto per intero prima di accertarsi che il codice sia UD e poterlo decodificare correttamente. Facendo l'esempio con dei simboli di codifica del tipo $a \to 0$, $b \to 01$ e $c \to 11$ e ricevendo il codice $C = 0111111\ldots$ allora verranno generati degli insiemi dei residui sempre uguali a $S_i = \{1\}$. Solo quando ricevo l'intero messaggio posso capire se questo sia veramente UD e cosi poterlo decodificare per associare correttamente i simboli giusti alle varie parole.

[5] Altri esempi nelle slide fornite dalla prof.

[6] Ovviamente, durante il confronto dell'insieme base S_0 con se stesso, i codici confrontati sono tra loro distinti. Altrimenti, confrontando un codice con se stesso, si otterrebbe un insieme vuoto, invalidando il risultato. Spiegato in altre parole, nel caso $a = bw$ **(Il residuo vecchio è prefisso della parola originale)** la parola originale a è più lunga. Essa inizia esattamente con la stringa b, e la parte che avanza è w. Invece, nel secondo caso $b = aw$ **(Il "viceversa": La parola originale è prefisso del residuo vecchio)** In questo scenario, è il residuo vecchio b ad essere più lungo. Esso inizia esattamente con la parola originale a, e la parte che avanza è w

- **Iterazione 3:** Per generare S_3 effettuo un confronto tra S_0 e S_2. Il primo elemento $eb \in S_2$ non è prefisso di nessuno (neanche il viceversa, ovvero $\nexists a \in S_0$ che faccia da prefisso ad eb). Il secondo elemento $cde \in S_2$ non è prefisso di nessuno, ma vale il viceversa cioè $c \in S_0$ è prefisso di cde; viene quindi generato il residuo $w = de$ l'unico nuovo elemento di $S_3 = \{de\}$.

- **Iterazione 4:** Per generare S_4 effettuo un confronto tra S_0 e S_3. L'unico elemento $de \in S_3$ è prefisso di $deb \in S_0$ che genera nuovamente un'insieme con un elemento $S_4 = \{b\}$.

- **Iterazione 5:** Per generare S_5 effettuo un confronto tra S_0 e S_4. L'unico elemento $b \in S_4$ è prefisso di due elementi di S_0, bad e $bbcde$, questo genera l'insieme $S_5 = \{ad, bcde\}$. L'algoritmo termina per la condizione $S_0 \cap S_n = \{ad\} \neq \emptyset$ affermando che il **codice non è UD**.

Per conferma di questa affermazione prendiamo il messaggio codificato *abbcdebad*, come si nota questo può essere espresso come:

1. $abb - c - deb - ad$

2. $a - bbcde - bad$

questo dimostra che non è un codice UD.

1.4.3 Esempio codice UD

Data la seguente tabella con vari codici di codifica:

Simbolo	**Codice A**	**Codice B**	**Codice C**	**Codice D**
Soleggiato	00	0	0	0
Nuvoloso	01	10	01	01
Piovoso	10	110	011	10
Annebbiato	11	1110	0111	1

sono presenti diversi tipi di codice. Partiamo dal **Codice D**, che non è unicamente decodificabile. Questo si osserva immediatamente: applicando l'algoritmo di Sardinas–Patterson, si ottiene subito un residuo pari a (1), segnale della presenza di ambiguità nella decodifica. Ora invece facciamo lo stesso esempio di prima ma con un codice UD. Prendiamo in esame il **codice C**, questo anche se lungo è UD, infatti seguendo i passi dell'algoritmo:

- Il passo iniziale è $S_0 = \{0, 01, 011, 0111\}$.

- **Iterazione 1:** Per generare S_1 prendo S_0 e lo confronto con se stesso cercando per ogni elemento un simbolo che sia il prefisso di un altro. Allora il primo insieme dei residui sarà $S_1 = \{1, 11, 111\}$ residui generati dal primo elemento (gli altri collassano essendo gli stessi).

- **Iterazione 2:** Per generare S_2 prendo l'insieme precedente e lo confronto con quello base, in questo caso non ci sono simboli che sono prefissi di altri. Allora l'insieme dei residui $S_2 = \emptyset$; l'algoritmo termina affermando che il codice è UD.

Per gli altri codici è facile provare che sono UD. Inoltre, il **codice A** è istantaneo.

1.4.4 Scelta del codice migliore.

Scegliere il codice più adatto a un certo ambito è una parte fondamentale della compressione dati. Cercando di minimizzare la lunghezza media dei messaggi trasmessi. Per ottenere la massima efficienza, il principio fondamentale è **assegnare le parole di codice più corte agli eventi più frequenti (più probabili)** e riservare le parole di codice più lunghe agli eventi più rari. Questo principio è strettamente legato al concetto di Entropia, che rappresenta il numero minimo di bit necessari in media per codificare un simbolo emesso da una sorgente. Consideriamo questi due eventi:

gli eventi rappresentano due stazioni meteo differenti. Vogliamo capire quale sia il codice migliore per rappresentare queste informazioni. Per la stazione di Roma tutti gli eventi hanno esattamente la stessa probabilità (1/4), un codice a lunghezza fissa come il Codice A (che usa

Roma	
Sorgente	**Probabilità**
Sole	1/4
Nuvoloso	1/4
Pioggia	1/4
Nebbia	1/4

Udine	
Sorgente	**Probabilità**
Sole	1/4
Nuvoloso	1/8
Pioggia	1/8
Nebbia	1/2

2 bit per ogni condizione: 00, 01, 10, 11) risulta essere molto appropriato ed efficiente visto che l'informazione principale che si trasporta è uno dei simboli. Invece, per la stazione di Udine si hanno delle probabilità differenti per i vari simboli per cui le informazioni possono essere compresse ed usare un codice a lunghezza fissa di 2 bit significherebbe sprecare spazio. In questo scenario è molto più conveniente usare un **codice a lunghezza variabile** così da poter risparmiare un po' di bit.

Per Udine i codici migliori potrebbero essere B o C; in entrambi i casi però l'evento più probabile dovrebbe avere un singolo bit e man mano che le probabilità diminuiscono assegnare i restanti simboli di codifica con una lunghezza maggiore. Per cui rivisitando il codice C ottimizzato per Udine ottengo:

Simbolo	Codice C
Annebbiato	0
Soleggiato	01
Nuvoloso	011
Piovoso	0111

facendo un piccolo conteggio inviando un messaggio al giorno per 100 giorni se usassi il codice A avrei usato 200 bit; usando il codice C userei 187 bit circa[7].

1.5 Codici istantanei

I codici istantanei sono un sottoinsieme dei codici unicamente decodificabili (UD); essi permettono di decodificare ogni parola di codice[8] senza dover attendere i simboli successivi.

Definizione 1.5. Un codice C si dice **a prefisso** se nessuna parola di codice è prefisso di un'altra parola di codice. In altre parole, nessun simbolo di codifica è prefisso di un altro simbolo dello stesso codice.

Definizione 1.6. Un codice C si dice **istantaneo** se ogni parola di codice può essere decodificata non appena viene letta, senza ambiguità e senza attendere simboli successivi.

Teorema 1.3. Un codice è istantaneo se e solo se è un codice a prefisso. Ogni codice a prefisso è unicamente decodificabile.

Questo risultato può essere dedotto, ad esempio, dal teorema di Sardinas-Patterson (**??**).

1.5.1 Ritardo di codifica

Come già accennato in precedenza la decodifica può avere un ritardo finito o infinito in dipendenza della **condizione di arresto**:

- Se $\exists i > 0, S_i = \emptyset$ allora il ritardo sarà finito, se $i = 1$ il codice è a prefisso[9].

  Per capire cosa si intende per **ritardo finito** facciamo riferimento all'algoritmo di Sardinas-Patterson. In questo algoritmo, ogni insieme S_i contiene i cosiddetti "residui". Questi residui vengono generati quando si cerca di costruire due diverse decodifiche per una stessa sequenza

[7] Sarebbero 50 bit per la nebbia, 50 per il sole, 37.5 per nuvoloso e altri 50 per piovoso. Si risparmia qualcosa rispetto a prima ed è anche possibile arrivare ad una compressione maggiore.

[8] Uso "parola di codice" per esprimere quello che prima è stato definito come "simbolo di codifica". Alla fine si intende la stessa cosa, ovvero un elemento dell'insieme C dei codici.

[9] Se appunto il primo insieme dei residui è già vuoto allora questo comporta un ritardo nullo. Per questo si parla di codici istantanei.

e una parola in codice risulta essere il prefisso di un'altra. I residui rappresentano la parte di stringa "avanzata" (ambigua) che deve essere risolta leggendo i simboli successivi. Se a un certo passo i l'insieme S_i risulta vuoto ($S_i = \emptyset$), significa che la catena di questi residui si interrompe e non si generano più nuove ambiguità. Di conseguenza, il ricevitore dovrà attendere al massimo la lettura di un numero limitato (e calcolabile) di simboli successivi prima che l'ambiguità si risolva e la parola possa essere decodificata con certezza.

Nell'esempio precedente (**??**) si nota come l'algoritmo si arresti ad un certo punto, quel punto indica proprio quanto si dovrà attendere prima di poter decodificare una parola.

- Se $\exists i, j > 0, S_i = S_j$ allora il ritardo è infinito. Come già spiegato, la presenza di un secondo insieme dei residui identico ad uno precedente comporta la presenza di un ciclo infinito. Questo accade perché l'ambiguità si propaga ciclicamente. Chi riceve potrebbe essere costretto ad aspettare l'intero messaggio prima di poter decodificare la prima parola e questa attesa appunto non ha un limite.

1.5.2 Esempio

Dall'esempio (**??**) proviamo a vedere se il codice A è istantaneo applicando sempre il solito algoritmo. Si nota subito come nessun simbolo di codifica è prefisso di un altro, da questo il primo insieme dei residui sarebbe già vuoto. Allora è ovvio che il codice A è a prefisso e questo implica che il codice A è istantaneo.

1.6 Disuguaglianza di Kraft-McMillan

Questa disuguaglianza stabilisce la condizione necessaria e sufficiente per l'esistenza di un codice a prefisso. Serve a capire **se è matematicamente possibile** creare un codice a prefisso partendo da un elenco di lunghezze desiderate per le parole in codice. La disuguaglianza però <u>non indica se un codice specifico sia a prefisso o meno</u> ma ci fa capire che se usassimo un codice con un numero di simboli d e con lunghezze specifiche allora quello può essere o meno un codice a prefisso. All'atto pratico ci aiuta ad escludere a monte i codici che non soddisfano tale disequazione i quali non sono sicuramente a prefisso.

Teorema 1.4. Consideriamo un alfabeto per la sorgente $S = \{s_1, s_2, \ldots, s_m\}$ e una alfabeto per la codifica $X = \{x_1, x_2, \ldots, x_m\}$. Dato il codice $\{c_1, c_2, \ldots, c_m\}$ ed essendo $\{l_1, l_2, \ldots, l_m\}$ le lunghezze delle rispettive parole di codifica c_i con $l_i \geq 1$. La condizione **necessaria e sufficiente** per l'esistenza di un codice istantaneo (o a prefisso), con una lunghezza delle parole di codifica specificate, è:

$$\sum_{i=1}^{m} d^{-l_i} \leq 1 \tag{1.6}$$

dove d è il numero **distinto** di simboli dell'alfabeto X usato per creare ogni elemento del codice[10].

- Se la disuguaglianza <u>non viene soddisfatta</u> allora non c'è modo di creare un codice istantaneo con le parole di codifica che si hanno a disposizione.
- Se la disuguaglianza <u>viene soddisfatta</u> allora c'è almeno un modo per creare un codice istantaneo con le parole di codifica che si hanno a disposizione.

Dimostrazione. Prima dimostriamo la **condizione sufficiente**. Indichiamo con $\{l_1, l_2, \ldots, l_m\}$ le lunghezze delle rispettive parole di codifica c_i che soddisfano l'equazione (**??**) allora cerchiamo di provare che esiste un codice a prefisso che le rispetta. Denotiamo con n_1 il numero di parole di codifica che hanno lunghezza pari a 1, con n_2 il numero di parole di codifica che hanno lunghezza pari a 2 e cosi via fino alla lunghezza massima L. Una volta ottenuti questi valori $\{n_1, n_2, \ldots, n_L\}$ possiamo affermare che $\sum_{i=1}^{L} n_i = m$, dove m è il numero di parole di codifica[11].

[10] Se il codice è binario allora $d = 2$ visto che sono presenti due simboli 0 e 1. Se il codice fosse formato da lettere o numeri allora d sarà uguale al numero di questi singoli simboli.

[11] Questo indica che la somma dei singoli raggruppamenti è uguale a tutti i gli elementi dell'insieme iniziale, ed è ovvio che sia così.

Una volta raggruppate tutte le parole di codifica con le stesse lunghezze possiamo scrivere:

$$\sum_{i=1}^{m} d^{-l_i} = \sum_{i=1}^{L} n_i d^{-i} \leq 1$$

questo ci permette di calcolare in una sola volta tutte le parole che condividono la stessa lunghezza[12]. Espandendo la somma si nota che moltiplicando per d ottengo:

$$n_1 + n_2 d^{-1} + n_3 d^{-2} + \ldots + n_L d^{-L+1} \leq d$$

è stato effettuato uno shift destro dei valori, e continuando a moltiplicare lo stesso valore:

$$\begin{aligned}
n_2 d^{-1} + n_3 d^{-2} + \ldots + n_L d^{-L+1} &\leq d - n_1 \\
n_3 d^{-1} + n_4 d^{-2} + \ldots + n_L d^{-L+2} &\leq d(d(d - n_1) - n_2) \\
n_4 d^{-1} + n_5 d^{-2} + \ldots + n_L d^{-L+3} &\leq (d(d(d - n_1) - n_2) - n_3) \\
&\vdots
\end{aligned}$$

otteniamo un insieme di disequazioni dove ogni raggruppamento n_i rappresenta un livello di un **albero d-ario**. Si possono quindi estrapolare le seguenti regole per ogni livello dell'albero:

$$\begin{aligned}
&\text{Livello } 1 \rightarrow n_1 \leq d \\
&\text{Livello } 2 \rightarrow n_2 \leq d(d - n_1) \\
&\text{Livello } 3 \rightarrow n_3 \leq d(d(d - n_1) - n_2) \\
&\vdots
\end{aligned}$$

questo indica che:

- Nel primo livello, che ospita le parole di lunghezza pari a 1, non si possono avere più parole dei rami totali a disposizione. Quindi è possibile prendere al massimo n_1 foglie (quelle con profondità 1).
- Nel secondo livello, che ospita le parole di lunghezza pari a 2, i rami liberi avanzati dal primo livello $(d - n_1)$ si moltiplicano per d al secondo livello. La formula garantisce che questi nuovi rami saranno sufficienti per ospitare le n_2 parole.
- E cosi via per tutti gli altri livelli.

Quindi, si costruisce un albero d-ario in cui i livelli di profondità rappresentano la lunghezza delle parole: al livello 1 troviamo i nodi per le parole di lunghezza 1, al livello 2 quelle di lunghezza 2, e così via fino alla profondità massima L. Ogni singola diramazione dell'albero rappresenta un simbolo specifico dell'alfabeto del codice (ad esempio, '0' o '1' per i codici binari). Le parole in codice si ottengono **concatenando i simboli dei rami** che si percorrono per raggiungere un determinato nodo.

Come si può notare in **??**, ogni volta che si assegna una parola a un nodo, questo diventa una 'foglia' e l'intero sotto-albero sottostante viene 'tagliato'. Le sue diramazioni non possono più essere usate, poiché percorrere i rami successivi significherebbe aggiungere ulteriori caratteri a una parola già conclusa, questo taglio garantisce matematicamente che la parola non possa mai diventare l'inizio (il prefisso) di una parola più lunga.

Figure 1: Rappresentazione albero d-ario Kraft-McMillan.

[12]Per capire questo passaggio basta pensare a come è fatta la somma. Raggruppando tutti gli elementi che hanno lo stesso esponente e moltiplicando il loro numero per d^{-i} dove i rappresenta proprio il livello si ottiene proprio la formula iniziale ma compressa. Possiamo fare un esempio per essere ancora più chiari. Si considerino le lunghezze 1,3,3 con simboli binari allora la disuguaglianza si dovrebbe scrivere come $2^{-1} + 2^{-3} + 2^{-3} = 0.75 \leq 1$ ma la stessa cosa si poteva benissimo scrivere anche come $1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 2 \cdot 2^{-3}$.

Scegliere un nodo significa 'consumare' una specifica porzione dell'albero. La disuguaglianza di Kraft-McMillan somma queste singole frazioni consumate per verificare che non superino il 100% dello spazio disponibile. Per esempio, in un albero binario, usare un nodo al primo livello consuma fisicamente metà dell'albero; nella formula, questa scelta pesa esattamente per 2^{-1}.

In conclusione, abbiamo dimostrato che, **se la disequazione è verificata**, allora, grazie alle regole stabilite per i singoli livelli, i rami non si esauriscono prima di aver posizionato tutte le parole; pertanto, il codice a prefisso può essere costruito assegnando ciascuna parola a una foglia.

Ora passiamo alla dimostrazione della **condizione necessaria**. Ipotizziamo che C sia un codice univocamente decodificabile (UD) con m parole di codifica con delle lunghezze rispettive di $\{l_1, l_2, \ldots, l_m\}$ su un alfabeto formato da d simboli. Consideriamo la quantità:

$$\left(\sum_{i=1}^{m} d^{-l_i}\right)^n = \left(d^{-l_1} + d^{-l_2} + \ldots + d^{-l_m}\right)^n$$

sviluppando questa potenza, otteniamo m^n termini. Ogni termine è il prodotto di un gruppo di n termini per cui ha la forma[13] $d^{-l_{i_1}-l_{i_2}-\ldots-l_{i_n}} = d^{-k}$, dove k è la somma delle lunghezze di n parole ($k = l_{i_1} + l_{i_2} + \ldots + l_{i_n}$). Il valore minimo di k è n (se tutte le parole sono lunghe 1) e il massimo è nL (se tutte sono lunghe L). Raggruppiamo tutti i termini che hanno lo stesso esponente k e otteniamo:

$$\sum_{k=n}^{nL} N_k \cdot d^{-k}$$

dove N_k rappresenta il numero di possibili sequenze (concatenazioni di parole) composte da n parole che hanno una lunghezza totale pari a k, ovvero il numero di volte che compare il termine d^{-k}. Poiché il codice deve essere **UD per ipotesi**, ogni concatenazione valida di parole deve corrispondere a una stringa univoca di simboli. Il numero massimo di stringhe distinte di lunghezza k creabili con d lettere è d^k.[14]Pertanto, N_k non può mai superare d^k ovvero:

$$N_k \leq d^k$$

applicando il caso limite ($N_k = d^k$) otteniamo:

$$\sum_{k=n}^{nL} d^k \cdot d^{-k} = \sum_{k=n}^{nL} 1 = nL - n + 1 \approx nL$$

ricapitolando vale:

$$\left(\sum_{i=1}^{m} d^{-l_i}\right)^n \leq \sum_{k=n}^{nL} d^k \cdot d^{-k} \leq nL$$

abbiamo quindi dimostrato che esiste un limite superiore pari proprio a nL (questo se tutte le parole nelle combinazioni sono lunghe L). Per far sì che la disequazione sia verificata è necessario che la somma sia minore o uguale a 1. Infatti, ponendo:

$$X = \sum_{i=1}^{m} d^{-l_i},$$

se per assurdo fosse $X > 1$, allora elevando alla n-esima si avrebbe $X^n \to +\infty$ per $n \to +\infty$. D'altra parte, dal lato opposto si ottiene una quantità che cresce al più linearmente in n[15]. Ne segue che, per n sufficientemente grande, la disuguaglianza non può più essere soddisfatta.

Si conclude quindi che deve necessariamente valere:

$$\sum_{i=1}^{m} d^{-l_i} \leq 1.$$

□

[13]Avendo tutti la stessa base quando viene svolta la potenza tutti i termini vengono raggruppati sotto una sola potenza.

[14]Si pensi al semplice e familiare caso binario le possibili combinazioni di simboli per k sarebbero proprio 2^k.

[15]La potenza va all'infinito molto più velocemente di una funzione lineare. Si dice che la funzione potenza sia di ordine superiore alla funzione lineare.

1.6.1 Proprietà

Un aspetto rilevante da considerare è distinguere il caso in cui la disequazione sia strettamente minore di 1 da quello in cui sia esattamente uguale a 1.

- Se la somma è **strettamente minore di 1**, il codice non è completo: ciò significa che, nella costruzione dell'albero associato al codice, rimangono ancora foglie libere, cioè combinazioni non assegnate ad alcuna parola di codice. In questo caso, quindi, esiste ancora spazio per aggiungere nuove parole senza violare la struttura del codice.
- Se la somma è **esattamente uguale a 1**, il codice è completo: tutte le possibilità compatibili con la struttura del codice sono state sfruttate. Nell'albero non rimangono più foglie utilizzabili per introdurre nuove parole senza compromettere le proprietà del codice stesso.

È importante osservare che questo non significa che ogni sequenza di simboli sia necessariamente una parola valida o una concatenazione decodificabile. Significa invece che, nel caso di uguaglianza, lo spazio disponibile è stato saturato completamente, mentre nel caso di stretta disuguaglianza rimane ancora spazio inutilizzato.

Consideriamo ora il confronto tra i codici B e C, presenti nel prossimo esercizio. Il codice B è costituito dalle parole:

$$0, \quad 100, \quad 110, \quad 111,$$

mentre il codice C è costituito dalle parole:

$$0, \quad 10, \quad 110, \quad 111.$$

Nel codice B rimane ancora una combinazione disponibile, ad esempio 101, che non è stata assegnata ad alcun simbolo. Questo mostra che il codice non sfrutta completamente tutte le possibilità offerte dalla sua struttura: infatti, si potrebbe aggiungere la parola 101 senza distruggere la proprietà di prefisso.

Nel codice C, invece, una situazione del genere non si verifica. La parola 10 è già presente nel codice e, proprio per questo, tutte le sequenze che iniziano con 10, come 100 e 101, non possono più essere usate come nuove parole di codice. Di conseguenza, il codice C non lascia più spazio per ulteriori assegnazioni ed è quindi completo.

Si noti però che la sequenza 101 **non è decodificabile** né nel codice B né nel codice C. Nel codice B ciò accade perché le uniche parole che iniziano con 1 sono 100, 110 e 111; di conseguenza, né 101 né una sequenza che inizi con tale prefisso possono essere riconosciute nel loro tratto iniziale come parole valide del codice (proprio per questo è incompleto). Nel codice C, invece, il prefisso 10 è effettivamente una parola di codice: per questo una sequenza che inizi con 101 non è immediatamente impossibile, ma richiede la ricezione di ulteriori simboli per verificare se la parte rimanente possa completare una concatenazione valida. La differenza essenziale è quindi che il codice B lascia alcune combinazioni inutilizzate senza poterle interpretare, mentre il codice C sfrutta in modo più completo la propria struttura.

1.6.2 Esempi

Data la tabella con i seguenti codici:

Simbolo	**Codice A**	**Codice B**	**Codice C**	**Codice D**	**Codice E**
s_1	00	0	0	0	0
s_2	01	100	10	100	10
s_3	10	110	110	110	110
s_4	11	111	111	11	11

grazie alla disuguaglianza di Kraft-McMillan possiamo capire quali tra questi potrebbero essere codici istantanei.

- Codice A:

$$\sum_{i=1}^{4} 2^{-2} = \sum_{i=2}^{2} 4 \cdot 2^{-i} = 1$$

ho usato entrambe le forme della dimostrazione. La disuguaglianza viene rispettata, si può quindi presumere che il codice A possa essere a prefisso visto che un codice binario con queste lunghezze può essere a prefisso. Per verificarlo bisogna usare l'algoritmo di Sardinas-Patterson (già verificato negli esempi precedenti).

- Codice B:
$$\sum_{i=1}^{4} 2^{-l_i} = 2^{-1} + 2^{-3} + 2^{-3} + 2^{-3} = \frac{7}{8} \leq 1$$
queste lunghezze sono valide per un codice a prefisso; da un'analisi più approfondita si può facilmente verificare che questo è un codice a prefisso.

- Codice C:
$$\sum_{i=1}^{4} 2^{-l_i} = 2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = \frac{8}{8} = 1$$
la condizione viene soddisfatta. Questo codice è a prefisso e sfrutta pienamente le possibili combinazioni delle varie lunghezze.

- Codice D:
$$\sum_{i=1}^{4} 2^{-l_i} = 2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = \frac{8}{8} = 1$$
questo codice soddisfa la condizione per essere a prefisso, come il codice C, ma al contrario di suddetto codice non è a prefisso. Infatti, solo per queste lunghezze viene assicurato che ci possa essere un codice a prefisso ma la scelta della concatenazione dei simboli è fondamentale per far si che questo codice sia veramente a prefisso. Si nota subito **che non è UD** visto che 11 è prefisso di 110 e il residuo è un'altra parola del codice.

- Codice E:
$$\sum_{i=1}^{4} 2^{-l_i} = 2^{-1} + 2^{-2} + 2^{-3} + 2^{-2} = 1 + \frac{1}{8}$$
questo codice non soddisfa la disuguaglianza, possiamo quindi concludere che non è istantaneo.

Fare esercizio direttamente qua per fare capire come si costruisce il codice

1.7 Esercizio

Si vuole codificare una sorgente con 9 simboli usando un codice a prefisso. L'alfabeto per le parole di codifica è 0,1,2 con delle lunghezze di 1,2,2,2,2,2,3,3,3.

Per prima cosa applichiamo il teorema di Kraft-McMillan per capire se con queste premesse può essere creato un codice a prefisso.

$$\sum_{i=1}^{9} 3^{l_i} = \sum_{k=1}^{3} N_k \cdot 3^{-i} = 1 \cdot 3^{-1} + 5 \cdot 3^{-2} + 3 \cdot 3^{-3} = 1$$

la disuguaglianza viene verificata si deduce che con questo alfabeto e queste lunghezze si può costruire un codice a prefisso.

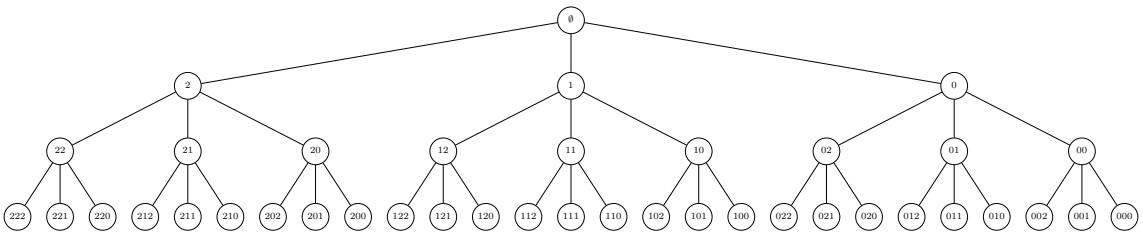

Figure 2: Albero ternario verticale di profondità 3